

Planificación del Proyecto

Sistema de Gestión Deportiva UCB

Replanificación de Sprints – Abril a Junio 2026

1. Estructura de los Equipos de Desarrollo

El proyecto se divide en 3 grupos de desarrollo con escalabilidad vertical, cada uno responsable de su propio backend y frontend.

Grupo	Enfoque	Responsabilidades Principales
Grupo 1 Identidad, Portal Web y Comunicación	Visibilidad institucional, autenticación y comunicación	– Sistema de autenticación OAuth2, Login y gestión de 4 roles – Portal Web Público (noticias, academias, jugadores destacados) – Módulo CMS y plantilla de comunicación a Marketing – Reactivación semestral de estudiantes (preservación historial) – Conexión con API de Validación Académica UCB
Grupo 2 Organización Deportiva y Torneos	Módulos internos de torneos y competencias	– Inscripción de equipos para Delegados (Torneo Intercarreras) – Algoritmo generación automática de fixtures y llaves – Prevención de colisiones de horarios (multidisciplinario) – Gestión de resultados y tablas de posiciones en tiempo real – Carga manual/visual de fixtures externos (asociaciones)
Grupo 3 Gestión de Recursos, Finanzas e Infraestructura	Control financiero, infraestructura y seguridad del sistema	– Control financiero de Academias (estados de cuenta, morosos) – Reserva de infraestructura (Coliseo y Cancha, calendario visual) – Generación automática de comprobantes PDF y envío SMTP – Integración con Vista SQL No Materializada de Tesorería – Middleware de Bitácora de Auditoría (Logs del sistema)

1.1 Alcances y Límites del Sistema

1.1.1 Alcance del Sistema (Qué SÍ hace)

El sistema tiene como objetivo **apoyar la gestión operativa y administrativa del deporte universitario**, cubriendo los siguientes alcances:

1.1.1.1 Gestión de usuarios y accesos

- Autenticación con cuenta institucional
- Gestión de roles y permisos (administrador, entrenador, delegado, estudiante/jugador)
- Control de acceso según rol

1.1.1.2 Gestión deportiva interna

- Registro de deportistas (estudiantes UCB y externos)
- Inscripción de deportistas a disciplinas y categorías
- Gestión de equipos y plantillas
- Inscripción de equipos a torneos (ej. Intercarreras)
- Gestión de torneos y competencias

- Generación automática de fixtures
- Registro de resultados de partidos
- Cálculo automático de tablas de posiciones

1.1.1.3 *Control administrativo básico*

- Gestión de reservas de canchas e infraestructura
- Generación automática de comprobantes de reserva en PDF
- Registro y verificación de pagos (manuales o integrados)
- Envío de notificaciones por correo institucional
- Mantenimiento de bitácora de auditoría del sistema

1.1.1.4 *Contenido web institucional*

- Portal web público informativo
- Publicación de noticias y comunicados
- Visualización de resultados, fixtures y tablas de posiciones
- Información general del club, disciplinas y actividades

1.1.1.5 *Registros de seguimiento simplificados*

- Control de asistencia por equipo y sesiones
- Reportes resumidos de rendimiento por equipo y torneo
- Registro de historial interno de cambios de equipo (solo informativo)

1.1.2. Límites del Sistema (Qué NO hace)

Para evitar desviaciones de alcance y mantener la viabilidad del proyecto, el sistema **NO contempla** los siguientes aspectos:

1.1.2.1 *Sin analítica ni KPIs avanzados*

- No se implementan dashboards de rendimiento
- No se generan indicadores estratégicos o métricas analíticas (KPIs)
- El sistema no realiza análisis comparativos ni predicciones

1.1.2.2 *Sin estadísticas individuales detalladas por jugador*

- No se registran estadísticas individuales por partido (goles, asistencias, minutos, etc.)
- No se realizan evaluaciones técnicas o físicas detalladas de jugadores
- No se manejan datos médicos, clínicos o sensibles

1.1.2.3 *Sin gestión de pases oficiales ni transferencias externas*

- El sistema **no gestiona pases oficiales** entre clubes o asociaciones
- No se coordinan transferencias con otras instituciones
- No se manejan montos, contratos ni documentación externa
- Solo se registra un **historial interno de cambios de equipo**, con fines informativos

1.1.2.4 *Sin integración obligatoria con sistemas externos*

- Las integraciones con sistemas académicos o de caja pueden operar con simulación o mecanismos básicos
- No se garantiza disponibilidad ni comportamiento de sistemas externos
- El sistema no depende críticamente de integraciones externas para operar

1.1.2.5 *Sin automatización de decisiones deportivas*

- El sistema no decide convocatorias, alineaciones ni sanciones
- No se bloquean inscripciones por criterios deportivos avanzados
- El sistema actúa como soporte administrativo, no como ente regulador

1.1.3. Consideración Final de Alcance

El sistema se enfoca exclusivamente en **la gestión administrativa y operativa del deporte universitario**, priorizando la simplicidad, viabilidad técnica y alineación con los requerimientos explícitos del proyecto.

Cualquier funcionalidad no incluida expresamente en este documento se considera **fuera del alcance**.

Beneficio inmediato para los grupos

Con esta sección:

- Todos saben **qué pueden y qué no pueden desarrollar**
- Evita “extras” de último momento
- Reduce trabajo innecesario
- Facilita coordinación entre grupos
- Te protege en revisión y defensa final

2. Planificación de Sprints

La planificación se ajusta a las fechas académicas del semestre, con hitos alineados a los parciales y el examen final.

Sprint 0 – Preparación y Planificación (6 – 12 de Abril de 2026: 1 semana)

Objetivo: Establecer bases estructurales, visuales y tecnológicas de forma colaborativa.

- Diseño UI/UX: Design System conjunto (colores, tipografías, botones, menús) y wireframes principales.
- Arquitectura de Datos: Diagramación colaborativa de la Base de Datos Relacional centralizada.
- Setup: Repositorio principal, arquitectura base de microservicios, estándares de código y configuración de entornos.

Sprint 1 – Desarrollo Core: Autenticación y Bases (13 – 26 de Abril de 2026: 2 semanas)

Objetivo: Desarrollar los cimientos de autenticación, inscripción de equipos y calendario de reservas.

ID HU	Historia de Usuario	Grupo	SP
HU-CORE-01	Autenticación con cuenta institucional (Login OAuth2)	G1	5
HU-CORE-02	Gestión de roles y permisos (4 roles)	G1	5
HU-GES-03	Registrar equipos para intercarreras	G2	3
HU-GES-04	Inscribir jugadores a equipos (datos simulados)	G2	5
HU-GES-18	Delegado inscribe equipo (acceso con credenciales)	G2	3
HU-GES-02	Gestión reservas de canchas con calendario dinámico y panel administrativo de las academias	G3	5
Total	6 Historias de Usuario		26 SP

Entregables Sprint 1:

- API de autenticación funcional con OAuth2
- Panel de gestión de roles (administrador, entrenador, delegado, estudiante/jugador)
- Módulo base de inscripción de equipos para Intercarreras
- Calendario dinámico de reservas (Coliseo y Cancha de Arquitectura)

Sprint 2 – Portal Web y Gestión Base (2° Parcial) (27 de Abril – 10 de Mayo de 2026: 2 semanas)

Objetivo: Maquetación del portal web público, CMS inicial y panel administrativo de academias.

ID HU	Historia de Usuario	Grupo	SP
HU-WEB-01	Página de inicio con noticias destacadas	G1	3
HU-WEB-02	Resultados de partidos recientes	G1	3
HU-WEB-03	Fixtures de torneos	G1	5

HU-WEB-04	Sección 'Inscríbete' con procedimientos	G1	2
HU-WEB-12	Diseño responsivo (Mobile-First)	G1	5
HU-GES-01	Registrar nuevo deportista en el sistema	G3	3
HU-GES-05	Verificación de pagos de academias (manual/simulado)	G3	5
HU-GES-17	Gestionar disciplinas deportivas (CRUD)	G3	2
HU-GES-06	Gestionar torneos y competencias	G2	3
HU-GES-07	Registrar resultados de partidos (interfaz básica)	G2	3
Total	10 Historias de Usuario		34 SP

Entregables Sprint 2:

- Portal web público maquetado con diseño responsivo
- CMS inicial (CRUD de academias, comunicados e historial del club)
- Panel administrativo de academias con estado de cuenta simulado
- Interfaz de registro de resultados y gestión de torneos

Sprint 3 – Lógica Compleja y Automatizaciones (Parte 1) (11 – 24 de Mayo de 2026: 2 semanas)

Objetivo: Automatizaciones clave, algoritmo de fixtures, generación de comprobantes PDF y contenido web complementario.

ID HU	Historia de Usuario	Grupo	SP
HU-CORE-09	Plantilla de comunicación a Marketing (CMS avanzado)	G1	3
HU-GES-15	Reactivar deportistas de semestres anteriores	G1	2
HU-WEB-08	Galería de fotos y videos del club	G1	3
HU-WEB-09	Horarios de entrenamiento por disciplina	G1	2
HU-WEB-10	Información y perfil de entrenadores	G1	2
HU-GES-08	Generar fixtures automáticamente (fase grupos / eliminación)	G2	8
HU-CORE-07	Prevención de colisiones de horarios (multidisciplinario)	G2	3
HU-GES-10	Registrar asistencia a entrenamientos	G2	3
HU-GES-11	Gestionar categorías deportivas	G2	2
HU-CORE-06	Generación comprobantes PDF + envío SMTP institucional	G3	5
HU-GES-14	Exportar reportes PDF/Excel por módulo	G3	3
Total	11 Historias de Usuario		36 SP

Entregables Sprint 3:

- CMS con plantilla de marketing (texto + imágenes, envío directo)
- Algoritmo automático de fixtures con prevención de colisiones
- Comprobantes de reserva en PDF enviados automáticamente por correo
- Contenido web complementario: galería, horarios, entrenadores
- Sistema de asistencia por entrenador y categorías deportivas

Sprint 4 – Lógica Compleja y Automatizaciones (Parte 2) (Nota Habilitación 3° Parcial) (26 de Mayo – 7 de Junio de 2026: 2 semanas)

Objetivo: Integraciones con sistemas institucionales (académico y de caja), tablas en tiempo real y funciones avanzadas.

Sprint de alta complejidad técnica. Se mitiga riesgo priorizando integración académica y pagos como historias obligatorias, dejando estadísticas avanzadas como ajustables

ID HU	Historia de Usuario	Grupo	SP
HU-CORE-03	Validación matrícula con sistema académico UCB (API booleana)	G1	8
HU-WEB-05	Perfiles de jugadores destacados	G1	5
HU-CORE-10	Búsqueda avanzada de deportistas (CI, carrera, disciplina)	G1	3
HU-WEB-07	Tablas de posiciones dinámicas	G2	3
HU-WEB-06	Calendario interactivo de eventos deportivos	G2	3
HU-GES-09	Jugadores habilitados en asociación (pases externos)	G2	5
HU-GES-16	Reporte resumido de datos del equipo por torneo	G2	5
HU-CORE-04	Integración pagos con Vista SQL No Materializada de Tesorería	G3	8
HU-CORE-05	Log de auditoría completo (Middleware)	G3	5
Total	9 Historias de Usuario		45 SP

Entregables Sprint 4:

- Microservicio de validación académica conectado al módulo de torneos
- Conciliación automática de pagos desde Vista SQL de Tesorería
- Tablas de posiciones y estadísticas actualizadas en tiempo real
- Bitácora de auditoría activa en todos los módulos
- Búsqueda avanzada y perfiles de jugadores destacados

Sprint 5 – Integración, Pruebas y Despliegue (Examen Final) (8 – 19 de Junio de 2026 (2 semanas)

Objetivo: Unir los módulos de los tres grupos, pruebas de integración/estrés y despliegue en producción.

Tareas de Integración Cruzada (Todos los Grupos)

- Integración de Autenticación: Grupo 1 provee OAuth2 para que Grupos 2 y 3 protejan sus APIs según rol

- Integración de Auditoría: Grupos 1 y 2 implementan el Middleware del Grupo 3 para registrar cambios en pagos, partidos e inscripciones
- Integración de Validación Académica (Grupos 1 y 2): Módulo de torneos conectado al microservicio de validación

Historias de Usuario – Sprint 5

ID HU	Historia de Usuario	Grupo	SP
HU-CORE-11	Despliegue en infraestructura UCB (Docker + Nginx)	G3	5
HU-CORE-12	Documentación API (Swagger/OpenAPI en /api-docs)	G3	3
HU-CORE-08	Dashboard de indicadores de gestión	G3	5
HU-GES-12	Control de participación por equipo	G2	5
HU-GES-13	Registrar historial interno de cambios de equipo	G2	5
HU-WEB-11	Blog / Artículos y consejos deportivos	G1	3
Total	6 Historias de Usuario		26 SP

Pruebas QA

- Simulación de tráfico para semana de inscripciones (Intercarreras): concurrencia 500+ usuarios
- Pruebas E2E de flujos críticos: pago → conciliación, inscripción → validación académica, reserva → PDF
- Corrección de incidencias identificadas en pruebas de usuario

Despliegue

- Migración de la base de datos al servidor de producción de la universidad
- Despliegue de microservicios y portal web en infraestructura UCB
- Carga de datos iniciales (disciplinas, configuración de canchas, usuarios administradores)
- Entrega final y capacitación a director, Don Víctor, entrenadores y delegados

Entregables Sprint 5:

- Sistema completo integrado y desplegado en producción
- Dashboard con KPIs deportivos y de gestión
- Documentación técnica (Swagger) y manual de usuario
- Evaluaciones físicas y control de pases entre clubes
- Blog público con consejos deportivos

3. Cronograma Resumido

Sprint	Fechas	Duración	HUs	SP	Hito
Sprint 0	6 – 12 Abr 2026	1 semana	–	Setup	Preparación y Planificación
Sprint 1	13 – 26 Abr 2026	2 semanas	6	26 SP	–
Sprint 2	27 Abr – 10 May 2026	2 semanas	10	34 SP	2° Parcial

Sprint 3	11 – 24 May 2026	2 semanas	11	36 SP	–
Sprint 4	26 May – 7 Jun 2026	2 semanas	9	45 SP	Nota Habilitación 3° Parcial
Sprint 5	8 – 19 Jun 2026	2 semanas	6	26 SP	Examen Final
TOTAL	6 Abr – 19 Jun 2026	11 semanas	42	167 SP	–

4. Product Backlog

El backlog completo del proyecto está organizado en tres épicas, una por grupo de desarrollo.

Épica 1: Identidad, Portal Web y Comunicación (Grupo 1)

ID	Historia de Usuario	Criterios de Aceptación	Prioridad	SP
HU-COR E-01	Como usuario, quiero autenticarme con mi cuenta institucional según mi rol	Login UCB / JWT / Redirección por rol	Alta	5
HU-COR E-02	Como administrador, quiero gestionar roles y permisos de acceso	Roles: admin, entrenador, delegado, estudiante / CRUD usuarios	Alta	5
HU-COR E-03	Como sistema, quiero validar matrícula de estudiantes con el sistema académico UCB	Integración API booleana / Validación por CI y semestre / Estado de validación	Alta	8
HU-COR E-09	Como director, quiero publicar contenido a Marketing con plantilla estructurada	Formulario título + texto + imágenes / Envío a Marketing / Notificación	Media	3
HU-COR E-10	Como administrativo, quiero buscar deportistas por múltiples criterios	Búsqueda por nombre, CI, disciplina, carrera / Resultados paginados	Media	3
HU-GES -15	Como administrativo, quiero reactivar deportistas de semestres anteriores	Buscar deportista existente / Confirmar reactivación / Mantener historial	Media	2
HU-WE B-01	Como visitante, quiero ver noticias destacadas del club	Home con noticias / Carrusel de imágenes / Enlaces a secciones	Alta	3
HU-WE B-02	Como estudiante, quiero consultar resultados de partidos recientes	Lista por disciplina / Fecha, rival y marcador / Filtro por torneo	Alta	3
HU-WE B-03	Como estudiante, quiero ver los fixtures de torneos	Calendario por disciplina / Por jornada / Jugado/pendiente	Alta	5

HU-WE B-04	Como visitante, quiero conocer el procedimiento para inscribirme	Sección 'Inscríbete' / Documentos requeridos / Link a formulario	Alta	2
HU-WE B-05	Como visitante, quiero ver los jugadores destacados del club	Perfiles con foto / Logros individuales / Participación en selecciones	Media	5
HU-WE B-08	Como visitante, quiero ver una galería de fotos y videos	Galería con thumbs / Modal de imágenes / Videos YouTube	Baja	3
HU-WE B-09	Como estudiante, quiero consultar horarios de entrenamiento	Horarios por disciplina y nivel / Día, hora, entrenador, lugar	Media	2
HU-WE B-10	Como visitante, quiero ver información de los entrenadores	Lista por disciplina / Foto, nombre, experiencia, títulos	Baja	2
HU-WE B-11	Como visitante, quiero leer artículos y consejos deportivos	Blog con artículos / Categorías / Fecha de publicación	Baja	3
HU-WE B-12	Como usuario, quiero que la web sea responsive para celular	Diseño adaptable / Menú colapsable / Imágenes optimizadas	Alta	5

Épica 2: Organización Deportiva y Torneos (Grupo 2)

ID	Historia de Usuario	Criterios de Aceptación	Prioridad	SP
HU-GE S-03	Como administrativo, quiero registrar equipos para intercarreras	Crear equipo por carrera / Asignar delegado / Lista de jugadores	Alta	3
HU-GE S-04	Como administrativo, quiero inscribir jugadores a equipos	Buscar por CI / Validar matrícula activa / Agregar a lista	Alta	5
HU-GE S-06	Como administrativo, quiero gestionar torneos y competencias	Crear torneo: nombre, disciplina, fechas, formato / Estado	Alta	3
HU-GE S-07	Como anotador, quiero registrar resultados de partidos	Seleccionar partido / Marcador final / Goleadores / Confirmar	Alta	3
HU-GE S-08	Como administrativo, quiero generar fixtures automáticamente	Formato grupos/eliminación / Fixture con fechas / Edición manual	Alta	8
HU-GE S-09	Como administrativo, quiero gestionar jugadores habilitados en asociación	Registrar con pase / Datos adicionales / Exportar formato BIS	Media	5
HU-GE S-10	Como entrenador, quiero registrar asistencia a entrenamientos	Lista por equipo / Presente/ausente / Observaciones	Media	3

HU-GE S-11	Como administrativo, quiero gestionar categorías deportivas	CRUD: sub-13, sub-15, juvenil, mayores / Rangos de edad	Media	2
HU-GE S-12	Como administrativo, quiero registrar la participación por equipo	Participación en torneos / Fecha	Media	5
HU-GE S-13	Como administrativo, quiero registrar cuando un jugador cambia de equipo dentro del sistema, para mantener trazabilidad histórica sin gestionar pases oficiales ni coordinaciones externas.	Registrar cambio interno de equipo / Equipo origen / Equipo destino / Fecha / Motivo	Baja	5
HU-GE S-16	Como administrativo, quiero registrar datos del equipo por torneo	Resultados, participantes y acumulado por torneo	Media	5
HU-GE S-17	Como administrativo, quiero gestionar disciplinas deportivas	CRUD disciplinas / Activar/desactivar / Asignar entrenadores	Alta	2
HU-GE S-18	Como delegado, quiero inscribir mi equipo al intercarreras	Acceso con credenciales / Seleccionar carrera / Lista con CI	Alta	3
HU-CO RE-07	Como administrativo, quiero evitar choques de horarios de partidos	Validar jugador en otros partidos / Misma hora en disciplina distinta / Alerta	Media	3
HU-WE B-06	Como estudiante, quiero consultar el calendario de eventos deportivos	Calendario mensual / Eventos: partidos, torneos / Vista de agenda	Media	3
HU-WE B-07	Como visitante, quiero ver las tablas de posiciones de torneos	Tabla actualizada / PJ, PG, PE, PP, GF, GC, Pts / Resaltar líderes	Alta	3

Épica 3: Gestión de Recursos, Finanzas e Infraestructura (Grupo 3)

ID	Historia de Usuario	Criterios de Aceptación	Prioridad	SP
HU-COR E-04	Como sistema, quiero recibir pagos desde caja UCB automáticamente	Vista SQL Tesorería / Actualizar estado en BD / Registrar transacción	Alta	8
HU-COR E-05	Como director, quiero que las acciones queden en log de auditoría	Tabla logs: usuario, fecha, acción, datos modificados / Filtros	Alta	5
HU-COR E-06	Como sistema, quiero enviar notificaciones automáticas por correo	Recordatorios de pago / Confirmación de reservas / SMTP	Media	5
HU-COR E-08	Como director, quiero generar reportes de indicadores de gestión	Dashboard / Inscritos por disciplina	Media	5

HU-COR E-11	Como sistema, quiero estar desplegado con alta disponibilidad	Docker + Nginx / Backup automático de BD	Alta	5
HU-COR E-12	Como desarrollador, quiero documentación de API actualizada	Swagger/OpenAPI en /api-docs / Descripción de endpoints	Media	3
HU-GES -01	Como administrativo, quiero registrar un nuevo deportista	Formulario: nombre, CI, disciplina, categoría, tipo / Guardar BD	Alta	3
HU-GES -02	Como administrativo, quiero gestionar reservas de canchas	Calendario disponibilidad / Registrar reserva / Comprobante PDF	Alta	5
HU-GES -05	Como administrativo, quiero verificar pagos de academias	Estado de pago visual / Marcado manual / Reporte de deudores	Alta	5
HU-GES -14	Como administrativo, quiero exportar reportes en PDF/Excel	Botón exportar por módulo / Excel datos / PDF presentación	Media	3

5. Métricas y Criterios de Éxito

Métrica	Objetivo
Velocity por sprint	30–45 Story Points
Cobertura pruebas unitarias	≥ 80%
Cobertura pruebas E2E (flujos críticos)	≥ 70%
Disponibilidad del sistema	99.5% (en horario administrativo)
Tiempo de respuesta API	< 500ms (p95)
Usuarios concurrentes soportados	500+ (picos de inscripción Intercarreras)
Tasa de adopción (administrativos)	100% después de capacitación

6. Recomendaciones Adicionales

- Product Owner: Lic. Fernando, con apoyo de Don Víctor para detalles operativos.
- Reuniones de seguimiento: Semanales de 30–45 minutos con el equipo de desarrollo.
- Pruebas tempranas: Involucrar a Don Víctor desde el Sprint 1 para validar módulos administrativos.
- Integraciones: Priorizar alianza con el área de Tecnología UCB para acceder a APIs del sistema académico y de caja antes del Sprint 4.
- Control de calidad: Cada grupo debe entregar con pruebas unitarias al final de cada sprint.

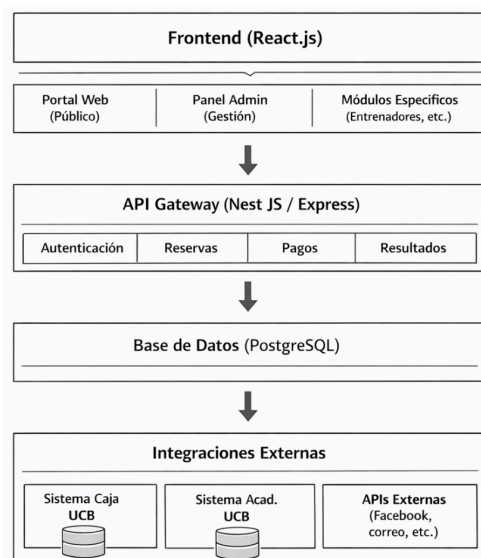
7. Stack Tecnológico Recomendado

Capa	Tecnología	Justificación
------	------------	---------------

Frontend	React 18 + TypeScript + Tailwind + Zustand	Componentización, rendimiento, diseño consistente
Backend	Node.js 20 + Express/Fastify (NestJS) + TypeScript + OAuth2 + Zod	Alto rendimiento I/O, ideal para APIs REST, consistencia con frontend
Base de Datos	PostgreSQL 15 + Prisma ORM + Redis	Relacional, robusta, migraciones tipadas, caché para sesiones/fixtures
Infraestructura	Docker + Nginx + GitHub Actions	Contenedores, proxy inverso, CI/CD automatizado
Monitorización	Sentry + Prometheus + Grafana	Errores en tiempo real, métricas de rendimiento
Integraciones	API REST (caja UCB, académico), SMTP (correos), S3/MinIO (archivos)	Desacoplamiento de módulos, escalabilidad horizontal

8. Arquitectura General

Se propone una arquitectura **basada en microservicios o módulos independientes**, pero con una base de datos centralizada. Esto permite que los tres grupos de trabajo desarrollen de forma paralela sin interferencias, garantizando la cohesión del sistema.



9. Integraciones Externas

Integración	Tecnología / Estrategia
Sistema de Caja UCB	Webhooks o consulta programada a API interna. Se requiere coordinar con el equipo de Tecnología UCB para obtener endpoints. Alternativa: archivos planos cifrados si no hay API disponible.
Sistema Académico UCB	API REST para validación de matrícula activa por CI. Se debe gestionar acceso con credenciales institucionales.
Notificaciones	Nodemailer + SMTP institucional para correos. Alternativa: servicio como Resend o SendGrid para mayor confiabilidad.
Almacenamiento de archivos	AWS S3 / MinIO (on-premise) para imágenes de jugadores, trofeos, galería.

10. Infraestructura y Despliegue

Componente	Opción recomendada
Entorno	Nube privada UCB o on-premise con virtualización. Alternativa: AWS / Google Cloud si hay presupuesto.
Contenedores	Docker + Docker Compose para desarrollo. Kubernetes si se requiere alta disponibilidad.
CI/CD	GitHub Actions / GitLab CI para pruebas automáticas y despliegue continuo.
Servidor Web	Nginx como reverse proxy y servidor de contenido estático.
Monitorización	Sentry (errores frontend/backend), Prometheus + Grafana (métricas de rendimiento).
Backups	pg_dump automatizado diario con retención de 30 días.

11. Herramientas de Desarrollo y Colaboración

Área	Herramienta	Propósito
Control de versiones	Git + GitHub / GitLab	Repositorio centralizado, manejo de ramas por grupo (portal-web, gestion-deportiva, core).
Gestión de proyecto	Jira / Trello / Notion	Seguimiento de historias de usuario, sprints y tareas.
Comunicación	Slack / Discord	Comunicación ágil entre grupos.
Documentación	Confluence / Notion	Documentación técnica, manuales de usuario, decisiones de arquitectura.
Testing	Jest (unitario), Playwright (e2e), Supertest (API)	Asegurar calidad antes de despliegues.

12. Consideraciones Especiales

12.1 Seguridad

- **HTTPS** obligatorio en todos los entornos.
- **CORS** restringido a dominios autorizados.
- **Rate limiting** en APIs públicas para evitar abusos.
- **Sanitización** de entradas para prevenir SQL injection y XSS.

12.2 Escalabilidad

- La arquitectura permite escalar horizontalmente los microservicios si la carga crece.
- La base de datos puede migrarse a un clúster gestionado si se requiere mayor disponibilidad.

12.3 Mantenibilidad

- Código modular con separación clara de responsabilidades.
- Pruebas automatizadas para minimizar regresiones.
- Documentación de API con Swagger accesible en /api-docs.

13. Resumen del Stack

Capa	Tecnología
------	------------

Frontend	React 18 + TypeScript + Tailwind + Zustand
Backend	Node.js 20 + Express + TypeScript + JWT + Zod
Base de Datos	PostgreSQL 15 + Prisma ORM + Redis
Infraestructura	Docker + Nginx + GitHub Actions
Monitorización	Sentry + Prometheus + Grafana
Integraciones	API REST (caja, académico), SMTP (correos), S3 (archivos)

14. Justificación de Elecciones

Requerimiento	Solución tecnológica
Portal web público con actualización dinámica	React + Tailwind, contenido consumido desde API
Integración con sistema académico y caja	API en Node.js con endpoints específicos, TypeScript para tipado seguro
Gestión de pagos y control de morosos	PostgreSQL + Prisma con modelo de transacciones; integración por webhooks o consulta programada
Generación automática de fixtures y tablas	Lógica en backend con algoritmos de torneo (round-robin, eliminación directa)
Reservas de canchas con comprobante	Endpoint API que genera PDF y envía por correo con Nodemailer
Roles y permisos diferenciados	JWT con claims de rol (admin, entrenador, anotador, delegado, estudiante)
Log de auditoría	Middleware en API que registra acciones en tabla de logs con usuario, fecha y datos modificados
Reportes PDF/Excel	Librería pdfkit / exceljs en backend, o generación desde frontend con librerías
Despliegue en infraestructura UCB	Docker + Nginx, compatible con entornos on-premise sin dependencia externa

15. Diagrama de Componentes (Resumen)



16. Modelos de Datos por Módulo

16.1 Módulo de Usuarios y Autenticación

```

-- Usuarios del sistema
CREATE TABLE usuarios (
  id SERIAL PRIMARY KEY,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  nombre_completo VARCHAR(150) NOT NULL,
  ci VARCHAR(20) UNIQUE,
  rol VARCHAR(50) NOT NULL, -- director, administrativo, entrenador, anotador, delegado,
  estudiante, padre
  activo BOOLEAN DEFAULT TRUE,
  ultimo_acceso TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Roles y permisos (RBAC)
CREATE TABLE roles (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) UNIQUE NOT NULL,
  descripcion TEXT
);
CREATE TABLE permisos (
  id SERIAL PRIMARY KEY,
  modulo VARCHAR(50) NOT NULL,
  accion VARCHAR(50) NOT NULL, -- crear, leer, actualizar, eliminar, exportar
  descripcion TEXT
);
CREATE TABLE roles_permisos (

```

```
rol_id INTEGER REFERENCES roles(id),
permiso_id INTEGER REFERENCES permisos(id),
PRIMARY KEY (rol_id, permiso_id)
);
-- Log de auditoría
CREATE TABLE logs_auditoria (
  id SERIAL PRIMARY KEY,
  usuario_id INTEGER REFERENCES usuarios(id),
  accion VARCHAR(100) NOT NULL,
  tabla_afectada VARCHAR(50),
  registro_id INTEGER,
  datos_anteriores JSONB,
  datos_nuevos JSONB,
  ip_origen INET,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

16.2 Módulo de Deportistas e Inscripciones

```
-- Deportistas (estudiantes UCB y externos)
CREATE TABLE deportistas (
  id SERIAL PRIMARY KEY,
  tipo VARCHAR(20) NOT NULL, -- estudiante_uchb, externo
  ci VARCHAR(20) UNIQUE NOT NULL,
  nombre_completo VARCHAR(150) NOT NULL,
  fecha_nacimiento DATE,
  genero VARCHAR(1),
  telefono VARCHAR(20),
  email VARCHAR(100),
  direccion TEXT,
  -- Campos específicos para estudiantes UCB
  carrera VARCHAR(100),
  semestre INTEGER,
  matricula_activa BOOLEAN DEFAULT FALSE,
  ultima_validacion_matricula TIMESTAMP,
  -- Campos generales
  talla_camiseta VARCHAR(10),
  fecha_ingreso DATE DEFAULT CURRENT_DATE,
  activo BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Disciplinas deportivas
CREATE TABLE disciplinas (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(50) UNIQUE NOT NULL, -- Fútbol, Básquetbol, Voleibol, Ajedrez
  descripcion TEXT,
  activo BOOLEAN DEFAULT TRUE,
  orden INTEGER DEFAULT 0
);
-- Inscripciones (relación deportista - disciplina)
CREATE TABLE inscripciones (
  id SERIAL PRIMARY KEY,
  deportista_id INTEGER REFERENCES deportistas(id),
  disciplina_id INTEGER REFERENCES disciplinas(id),
  categoria VARCHAR(50), -- sub-13, sub-15, juvenil, mayor, competitivo, academias,
  cursos_gratuitos
  nivel VARCHAR(50), -- principiante, intermedio, avanzado, competitivo
  fecha_inscripcion DATE DEFAULT CURRENT_DATE,
  fecha_baja DATE,
  motivo_baja TEXT,
  estado VARCHAR(20) DEFAULT 'activo', -- activo, retirado, suspendido, ausente
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

    UNIQUE(deportista_id, disciplina_id, categoria, estado)
);
-- Historial de inscripciones por semestre/año
CREATE TABLE historial_inscripciones (
    id SERIAL PRIMARY KEY,
    deportista_id INTEGER REFERENCES deportistas(id),
    disciplina_id INTEGER REFERENCES disciplinas(id),
    semestre VARCHAR(20), -- 2025-1, 2025-2
    categoria VARCHAR(50),
    participo_torneos BOOLEAN DEFAULT FALSE,
    logros TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

16.3 Módulo de Pagos (Academias)

```

-- Conceptos de pago
CREATE TABLE conceptos_pago (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL, -- Matrícula, Mensualidad, Derecho de cancha, Pase
    monto DECIMAL(10,2) NOT NULL,
    periodicidad VARCHAR(20), -- única, mensual, anual
    disciplina_id INTEGER REFERENCES disciplinas(id),
    activo BOOLEAN DEFAULT TRUE
);
-- Pagos registrados
CREATE TABLE pagos (
    id SERIAL PRIMARY KEY,
    deportista_id INTEGER REFERENCES deportistas(id),
    concepto_id INTEGER REFERENCES conceptos_pago(id),
    monto DECIMAL(10,2) NOT NULL,
    mes INTEGER, -- 1-12 (para mensualidades)
    anio INTEGER,
    fecha_pago DATE NOT NULL,
    comprobante VARCHAR(100), -- número de recibo o referencia
    origen VARCHAR(20) DEFAULT 'manual', -- manual, integracion_caja
    estado VARCHAR(20) DEFAULT 'confirmado', -- confirmado, pendiente, anulado
    observaciones TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Planilla de pagos por academia (vista o tabla de control)
CREATE TABLE planilla_pagos_academia (
    id SERIAL PRIMARY KEY,
    deportista_id INTEGER REFERENCES deportistas(id),
    anio INTEGER NOT NULL,
    matricula_pagada BOOLEAN DEFAULT FALSE,
    mes_1_pagado BOOLEAN DEFAULT FALSE, -- marzo
    mes_2_pagado BOOLEAN DEFAULT FALSE, -- abril
    mes_3_pagado BOOLEAN DEFAULT FALSE, -- mayo
    mes_4_pagado BOOLEAN DEFAULT FALSE, -- junio
    mes_5_pagado BOOLEAN DEFAULT FALSE, -- julio
    mes_6_pagado BOOLEAN DEFAULT FALSE, -- agosto
    mes_7_pagado BOOLEAN DEFAULT FALSE, -- septiembre
    mes_8_pagado BOOLEAN DEFAULT FALSE, -- octubre
    mes_9_pagado BOOLEAN DEFAULT FALSE, -- noviembre
    total_pagado DECIMAL(10,2) DEFAULT 0,
    saldo_pendiente DECIMAL(10,2) DEFAULT 0,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

16.4 Módulo de Reservas de Canchas

```

-- Espacios deportivos
CREATE TABLE espacios (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL, -- Coliseo, Cancha Arquitectura
  ubicacion TEXT,
  capacidad INTEGER,
  horario_apertura TIME,
  horario_cierre TIME,
  activo BOOLEAN DEFAULT TRUE
);
-- Reservas de canchas
CREATE TABLE reservas (
  id SERIAL PRIMARY KEY,
  espacio_id INTEGER REFERENCES espacios(id),
  solicitante_id INTEGER REFERENCES usuarios(id),
  deportista_id INTEGER REFERENCES deportistas(id),
  fecha DATE NOT NULL,
  hora_inicio TIME NOT NULL,
  hora_fin TIME NOT NULL,
  disciplina_id INTEGER REFERENCES disciplinas(id),
  motivo VARCHAR(200),
  estado VARCHAR(20) DEFAULT 'pendiente', -- pendiente, confirmada, cancelada, finalizada
  comprobante_pdf TEXT, -- ruta o URL del PDF generado
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Horarios disponibles (plantilla semanal)
CREATE TABLE horarios_disponibles (
  id SERIAL PRIMARY KEY,
  espacio_id INTEGER REFERENCES espacios(id),
  dia_semana INTEGER, -- 0=Domingo, 1=Lunes, ..., 6=Sábado
  hora_inicio TIME,
  hora_fin TIME,
  disponible BOOLEAN DEFAULT TRUE
);

```

16.5 Módulo de Torneos y Competencias

```

-- Torneos
CREATE TABLE torneos (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  disciplina_id INTEGER REFERENCES disciplinas(id),
  tipo VARCHAR(50) NOT NULL, -- intercarreras, externo, amistoso
  formato VARCHAR(50) NOT NULL, -- liga, eliminacion_directa, grupos_eliminacion
  categoria VARCHAR(50), -- sub-13, sub-15, juvenil, mayor
  temporada VARCHAR(20), -- 2025
  fecha_inicio DATE,
  fecha_fin DATE,
  estado VARCHAR(20) DEFAULT 'planificado', -- planificado, en_curso, finalizado, cancelado
  reglas TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Equipos
CREATE TABLE equipos (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  carrera VARCHAR(100), -- para intercarreras
  torneo_id INTEGER REFERENCES torneos(id),
  entrenador_id INTEGER REFERENCES usuarios(id),
  delegado_id INTEGER REFERENCES usuarios(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

-- Jugadores por equipo (plantilla)
CREATE TABLE equipo_jugadores (
  id SERIAL PRIMARY KEY,
  equipo_id INTEGER REFERENCES equipos(id),
  deportista_id INTEGER REFERENCES deportistas(id),
  numero_camiseta INTEGER,
  posicion VARCHAR(50),
  fecha_inscripcion DATE DEFAULT CURRENT_DATE,
  fecha_baja DATE,
  habilitado BOOLEAN DEFAULT TRUE
);
-- Partidos
CREATE TABLE partidos (
  id SERIAL PRIMARY KEY,
  torneo_id INTEGER REFERENCES torneos(id),
  equipo_local_id INTEGER REFERENCES equipos(id),
  equipo_visitante_id INTEGER REFERENCES equipos(id),
  fecha DATE NOT NULL,
  hora TIME NOT NULL,
  espacio_id INTEGER REFERENCES espacios(id),
  jornada INTEGER,
  goles_local INTEGER DEFAULT 0,
  goles_visitante INTEGER DEFAULT 0,
  estado VARCHAR(20) DEFAULT 'programado', -- programado, en_curso, finalizado, suspendido
  acta_url TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Estadísticas individuales por partido
CREATE TABLE estadisticas_partido (
  id SERIAL PRIMARY KEY,
  partido_id INTEGER REFERENCES partidos(id),
  deportista_id INTEGER REFERENCES deportistas(id),
  equipo_id INTEGER REFERENCES equipos(id),
  goles INTEGER DEFAULT 0,
  asistencias INTEGER DEFAULT 0,
  tarjetas_amarillas INTEGER DEFAULT 0,
  tarjetas_rojas INTEGER DEFAULT 0,
  minutos_jugados INTEGER DEFAULT 0,
  observaciones TEXT
);
-- Fixtures (generados automáticamente)
CREATE TABLE fixtures (
  id SERIAL PRIMARY KEY,
  torneo_id INTEGER REFERENCES torneos(id),
  jornada INTEGER,
  fecha DATE,
  partidos JSONB, -- lista de IDs de partidos ordenados
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Tabla de posiciones (calculada)
CREATE TABLE tabla_posiciones (
  id SERIAL PRIMARY KEY,
  torneo_id INTEGER REFERENCES torneos(id),
  equipo_id INTEGER REFERENCES equipos(id),
  partidos_jugados INTEGER DEFAULT 0,
  partidos_ganados INTEGER DEFAULT 0,
  partidos_empatados INTEGER DEFAULT 0,
  partidos_perdidos INTEGER DEFAULT 0,
  goles_favor INTEGER DEFAULT 0,
  goles_contra INTEGER DEFAULT 0,
  puntos INTEGER DEFAULT 0,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

16.6 Módulo de Jugadores Habilitados (Asociaciones)

```
-- Jugadores con pase (competencias externas)
CREATE TABLE jugadores_pase (
  id SERIAL PRIMARY KEY,
  deportista_id INTEGER REFERENCES deportistas(id),
  numero_pase VARCHAR(50) UNIQUE,
  asociacion VARCHAR(100), -- Asociación Departamental de Básquetbol, etc.
  fecha_habilitacion DATE,
  fecha_vencimiento DATE,
  posicion_juego VARCHAR(50),
  alcance_brazo DECIMAL(5,2), -- en cm
  peso DECIMAL(5,2), -- en kg
  historial_clubes TEXT, -- JSON con historial de transferencias
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Transferencias entre clubes
CREATE TABLE transferencias (
  id SERIAL PRIMARY KEY,
  jugador_pase_id INTEGER REFERENCES jugadores_pase(id),
  club_origen VARCHAR(100),
  club_destino VARCHAR(100),
  monto DECIMAL(10,2),
  fecha_transferencia DATE,
  documento_url TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

16.7 Módulo de Evaluaciones y Rendimiento

```
-- Evaluaciones físicas y técnicas
CREATE TABLE evaluaciones (
  id SERIAL PRIMARY KEY,
  deportista_id INTEGER REFERENCES deportistas(id),
  entrenador_id INTEGER REFERENCES usuarios(id),
  disciplina_id INTEGER REFERENCES disciplinas(id),
  fecha_evaluacion DATE NOT NULL,
  tipo VARCHAR(50) NOT NULL, -- fisica, tecnica, tactica, psicologica
  -- Datos físicos
  peso DECIMAL(5,2),
  altura DECIMAL(5,2),
  porcentaje_grasa DECIMAL(5,2),
  alcance_brazo DECIMAL(5,2),
  -- Evaluación técnica (escala 1-10)
  tecnica_puntaje INTEGER,
  tactica_puntaje INTEGER,
  fisico_puntaje INTEGER,
  psicologico_puntaje INTEGER,
  -- Observaciones
  fortalezas TEXT,
  debilidades TEXT,
  recomendaciones TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

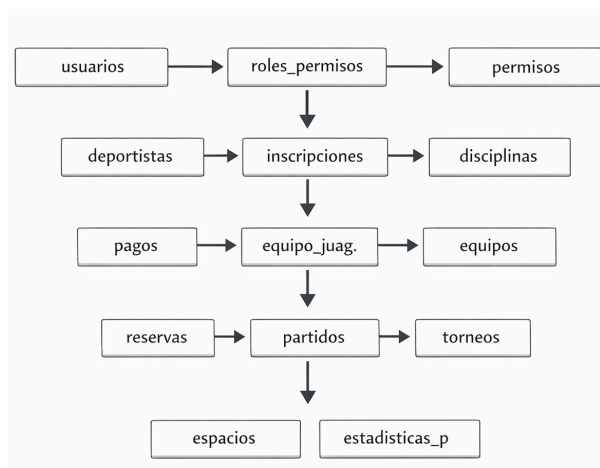
-- Historial de rendimiento por temporada
CREATE TABLE rendimiento_temporada (
  id SERIAL PRIMARY KEY,
  deportista_id INTEGER REFERENCES deportistas(id),
  temporada VARCHAR(20),
  disciplina_id INTEGER REFERENCES disciplinas(id),
  total_partidos INTEGER DEFAULT 0,
  total_goles INTEGER DEFAULT 0,
```

```
total_asistencias INTEGER DEFAULT 0,  
promedio_rendimiento DECIMAL(3,2), -- 0-10  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

16.8 Módulo de Contenido Web

```
-- Noticias y artículos  
CREATE TABLE contenido_web (  
  id SERIAL PRIMARY KEY,  
  tipo VARCHAR(50) NOT NULL, -- noticia, articulo, consejo  
  titulo VARCHAR(200) NOT NULL,  
  contenido TEXT NOT NULL,  
  imagen_destacada TEXT,  
  autor_id INTEGER REFERENCES usuarios(id),  
  fecha_publicacion DATE,  
  visible BOOLEAN DEFAULT TRUE,  
  destacado BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Categorías de contenido  
CREATE TABLE categorias_contenido (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(50) UNIQUE NOT NULL  
);  
  
CREATE TABLE contenido_categorias (  
  contenido_id INTEGER REFERENCES contenido_web(id),  
  categoria_id INTEGER REFERENCES categorias_contenido(id),  
  PRIMARY KEY (contenido_id, categoria_id)  
);  
  
-- Jugadores destacados  
CREATE TABLE jugadores_destacados (  
  id SERIAL PRIMARY KEY,  
  deportista_id INTEGER REFERENCES deportistas(id),  
  temporada VARCHAR(20),  
  logros TEXT,  
  reconocimientos TEXT,  
  foto_perfil TEXT,  
  orden INTEGER DEFAULT 0,  
  activo BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Galería multimedia  
CREATE TABLE galeria (  
  id SERIAL PRIMARY KEY,  
  tipo VARCHAR(20) NOT NULL, -- foto, video  
  titulo VARCHAR(200),  
  url VARCHAR(500) NOT NULL,  
  thumbnail_url VARCHAR(500),  
  evento VARCHAR(100),  
  fecha_subida DATE DEFAULT CURRENT_DATE,  
  publicado BOOLEAN DEFAULT TRUE  
);
```

16.9 Diagrama de Relaciones (Resumen)



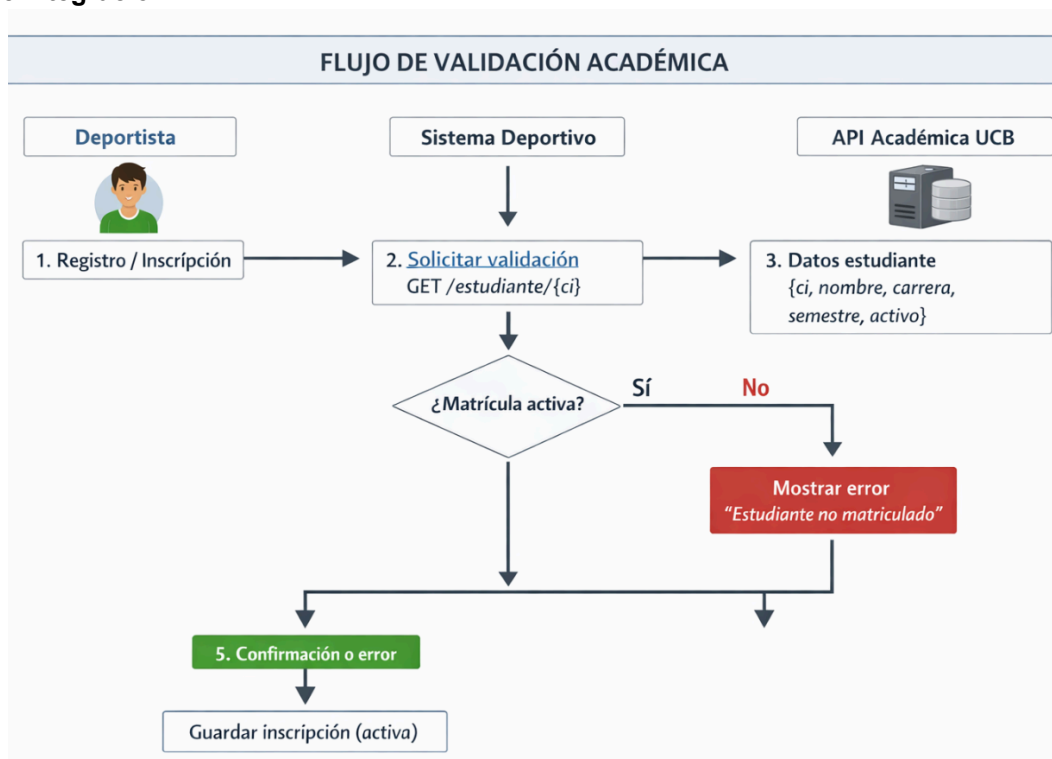
17. Diagramas de Flujo de Integración

17.1 Integración con Sistema Académico UCB

Propósito

Validar automáticamente que un estudiante está matriculado activamente en el semestre en curso al momento de inscribirse a intercarreras o cursos gratuitos.

Flujo de Integración



Detalle Técnico

Elemento	Especificación
Endpoint esperado	GET /api/academica/estudiante/{ci}
Parámetros	CI (carnet de identidad)

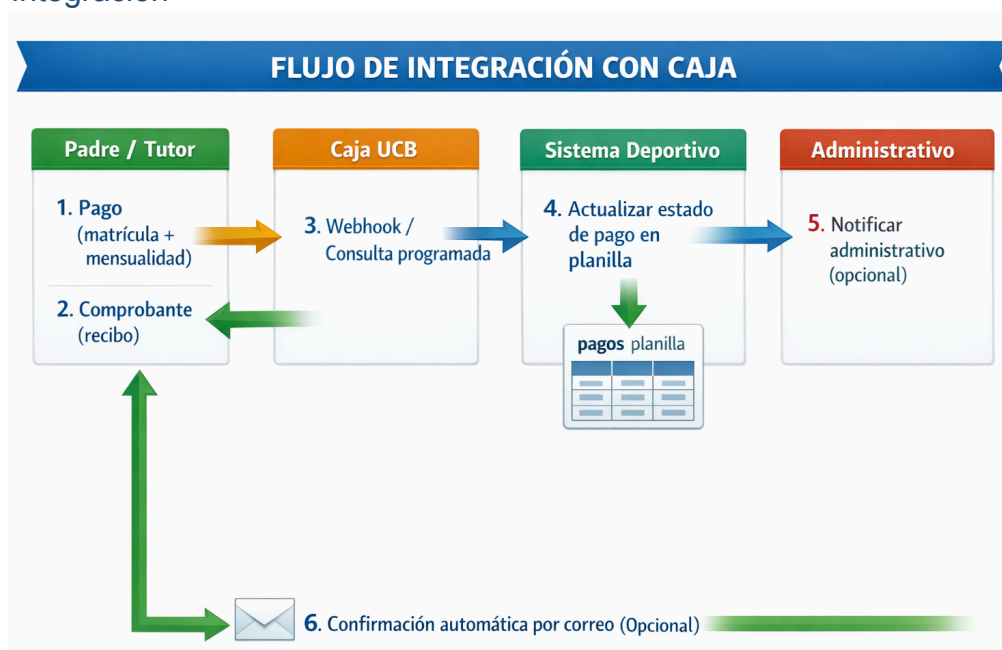
Respuesta exitosa	{ "ci": "1234567", "nombre": "Juan Perez", "carrera": "Ingeniería de Sistemas", "semestre": 5, "matricula activa": true, "fecha vencimiento": "2025-12-31" }
Frecuencia	En tiempo real al momento de inscripción
Mecanismo fallback	Si API no disponible, permitir inscripción con alerta y validación posterior asíncrona
Cache	Redis TTL 24 horas para reducir llamadas repetidas

17.2 Integración con Sistema de Caja UCB

Propósito

Reflejar automáticamente los pagos realizados en la caja universitaria en el sistema deportivo, eliminando la verificación manual y los conflictos por pagos no registrados.

Flujo de Integración

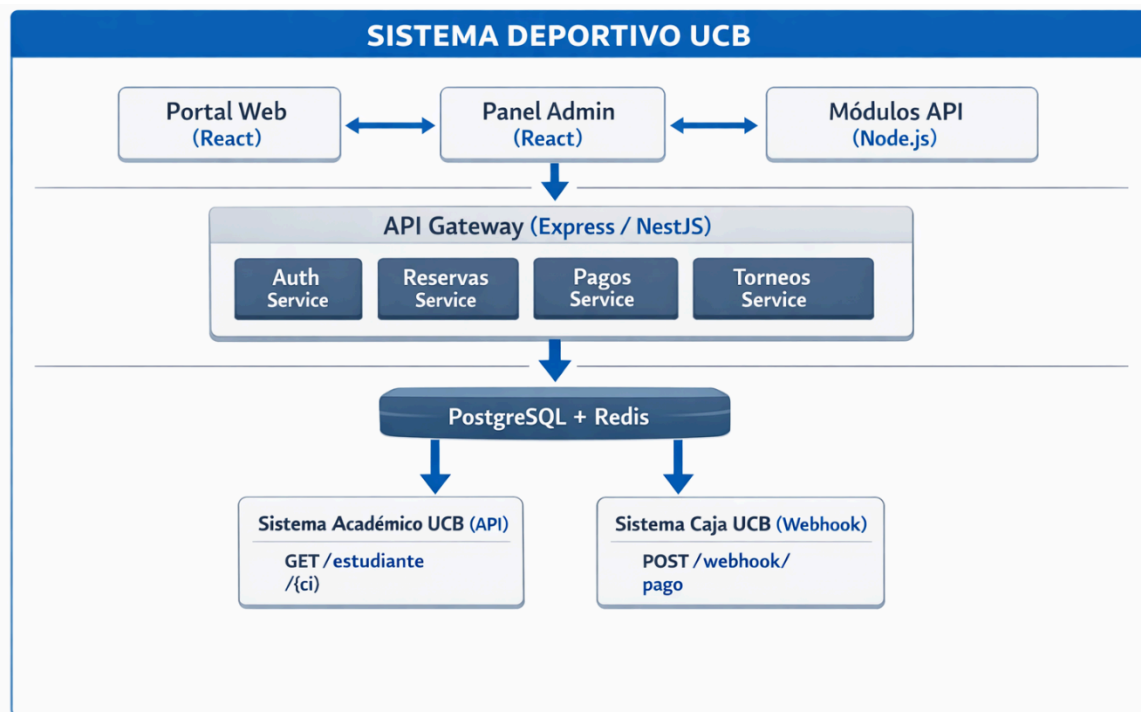


Estrategia de Integración

Estrategia	Descripción	Ventajas	Desventajas
Webhook	Caja UCB notifica al sistema deportivo inmediatamente después de cada pago	Tiempo real, baja latencia	Requiere que caja implemente llamada HTTP
Consulta programada	Sistema deportivo consulta pagos pendientes cada X minutos	No requiere cambios en la caja	Latencia de hasta X minutos
Archivo de intercambio	Caja exporta archivo plano cifrado, sistema lo consume periódicamente	Simple, seguro	No es tiempo real

Recomendación: Implementar Webhook como primario y consulta programada como fallback para garantizar disponibilidad.

17.3 Diagrama de Componentes con Integraciones



18. Estrategia de Pruebas por Grupo

18.1 Grupo 1 – Portal Web Público

Objetivo

Garantizar que la página web sea funcional, accesible, responsive y que el contenido se visualice correctamente en diferentes dispositivos.

Tipos de Pruebas

Tipo	Herramientas	Alcance	Criterio de Aceptación
Pruebas Unitarias	Jest + React Testing Library	Componentes individuales (Header, Footer, Card, Calendar)	Cobertura $\geq 80\%$
Pruebas de Integración	React Testing Library	Interacciones entre componentes y consumo de API	Flujos críticos funcionan
Pruebas E2E	Playwright / Cypress	Flujo completo: navegación, visualización de resultados, fixtures, calendario	Escenarios críticos automatizados
Pruebas Responsive	Chrome DevTools + BrowserStack	Dispositivos: móvil (iPhone, Android), tablet, desktop	Sin errores de layout en tamaños definidos
Pruebas de Accesibilidad	axe-core / Lighthouse	WCAG 2.1 nivel AA	Puntaje Lighthouse ≥ 90
Pruebas de Rendimiento	Lighthouse, WebPageTest	Tiempo de carga, LCP, FID	LCP $< 2.5s$, FID $< 100ms$
Pruebas de Usuario	Usuarios reales (estudiantes)	Navegación, comprensión de información	Feedback positivo $\geq 80\%$

Casos de Prueba Prioritarios

ID	Escenario	Resultado Esperado
WEB-TC-01	Cargar página de inicio	Noticias destacadas visibles, carrusel funcional
WEB-TC-02	Navegar a Resultados	Lista de resultados recientes por disciplina
WEB-TC-03	Navegar a Fixtures	Calendario de partidos por torneo
WEB-TC-04	Acceder desde móvil	Menú colapsable, contenido legible sin zoom
WEB-TC-05	Consultar tabla de posiciones	Tabla actualizada con datos correctos
WEB-TC-06	Ver perfil de jugador destacado	Foto, logros, información visible
WEB-TC-07	Navegar a Insíbete	Procedimientos claros, enlaces funcionales

18.2 Grupo 2 – Gestión Deportiva Administrativa

Objetivo

Validar que los módulos administrativos funcionan correctamente, los datos se persisten y los procesos de negocio se cumplen.

Tipos de Pruebas

Tipo	Herramientas	Alcance	Criterio de Aceptación
Pruebas Unitarias	Jest + Supertest	Servicios: registro, pagos, reservas, torneos	Cobertura $\geq$ 80%
Pruebas de Integración	Jest + Supertest + PostgreSQL test	Endpoints API, validaciones de negocio	Todos los endpoints críticos probados
Pruebas Funcionales	Postman / Bruno	Flujos completos: inscripción → pago → verificación	Escenarios positivos y negativos
Pruebas de Regresión	Jest + Supertest	Ejecutar suite completa después de cambios	Sin fallos en funcionalidades existentes
Pruebas de Usuario (UAT)	Director, Don Víctor, entrenadores	Módulos: inscripciones, pagos, reservas, intercarreras	Aprobación del usuario final

Casos de Prueba Prioritarios

ID	Escenario	Resultado Esperado
GES-TC-01	Registrar nuevo deportista estudiante UCB	Datos guardados, validación académica ejecutada
GES-TC-02	Registrar deportista externo (academia)	Datos guardados, sin validación académica
GES-TC-03	Registrar pago manual de mensualidad	Estado actualizado en planilla, saldo pendiente recalcula
GES-TC-04	Generar fixture para validar HU-GES-08 todos contra todos (4 equipos)	Fixture generado con 6 partidos, jornadas balanceadas
GES-TC-05	Registrar resultado de partido	Tabla de posiciones actualizada automáticamente
GES-TC-06	Crear reserva de cancha	Comprobante PDF generado, espacio bloqueado en calendario
GES-TC-07	Inscribir equipo a intercarreras	Equipo registrado, jugadores validados por matrícula
GES-TC-08	Intentar inscribir jugador sin matrícula activa	Error mostrado, inscripción no permitida
GES-TC-09	Exportar reporte de pagos a Excel	Archivo descargable con datos correctos

18.3 Grupo 3 – Núcleo y Backend

Objetivo

Garantizar la estabilidad, seguridad, rendimiento y confiabilidad del backend, integraciones y datos.

Tipos de Pruebas

Tipo	Herramientas	Alcance	Criterio de Aceptación
Pruebas Unitarias	Jest + Supertest	Controladores, servicios, middlewares, utilidades	Cobertura $\geq$ 85%
Pruebas de Integración	Jest + Supertest + PostgreSQL test	Endpoints con autenticación, roles, permisos	100% de endpoints críticos
Pruebas de API	Postman / Bruno + Newman	Colección completa de endpoints	100% de endpoints documentados probados
Pruebas de Seguridad	OWASP ZAP, npm audit	Autenticación JWT, CORS, rate limiting, SQL injection	Sin vulnerabilidades críticas o altas
Pruebas de Carga	k6 / Artillery	Endpoints críticos: Login, inscripción, resultados	500 usuarios concurrentes, respuesta < 500ms
Pruebas de Integración Externa	Mock APIs / Sandbox	Integración con sistema académico y caja	Fallback funciona si API externa falla
Pruebas de Auditoría	Pruebas manuales	Logs de auditoría registran acciones críticas	Todas las modificaciones importantes tienen log
Pruebas de Backup/Restore	pg_restore simulado	Recuperación de base de datos desde backup	Backup restaurable en < 30 minutos

Casos de Prueba Prioritarios

ID	Escenario	Resultado Esperado
CORE-TC-01	Login con credenciales válidas	Token JWT retornado, rol asignado
CORE-TC-02	Acceso a endpoint sin autenticación	HTTP 401 Unauthorized
CORE-TC-03	Usuario entrenador accede a endpoint de director	HTTP 403 Forbidden
CORE-TC-04	Validación de matrícula con CI válido	Retorna activo=true, datos correctos
CORE-TC-05	Validación de matrícula con CI inválido	Retorna activo=false, error amigable
CORE-TC-06	Webhook de pago recibido desde caja	Estado de pago actualizado, registro en logs
CORE-TC-07	API académica no disponible	Fallback a validación manual, alerta registrada
CORE-TC-08	Modificar resultado de partido	Log de auditoría registra cambio (antes/después)
CORE-TC-09	Rate limiting excedido	HTTP 429 Too Many Requests
CORE-TC-10	500 usuarios concurrentes en Login	Tiempo respuesta p95 < 500ms

18.4 Estrategia General de Pruebas

Entornos

Entorno	Propósito	Datos	Acceso
Desarrollo (Dev)	Desarrollo y pruebas unitarias	Datos maquetados	Equipo técnico
Pruebas (QA)	Pruebas integradas, funcionales, carga	Datos anonimizados (copiados de producción)	Equipo QA + Product Owner
Staging / Pre-producción	Validación final, UAT	Datos reales anonimizados	Product Owner, usuarios clave
Producción	Operación real	Datos reales	Usuarios finales

Frecuencia de Pruebas

Tipo	Frecuencia	Responsable
Unitarias	Por cada commit	Desarrollador
Integración	Por cada PR	Desarrollador + CI
Funcionales	Diario (automático)	CI/CD pipeline
Regresión	Antes de cada release	QA
Carga	Antes de release mayor	QA + DevOps
Seguridad	Mensual	DevOps
UAT	Al final de cada sprint	Product Owner + usuarios

Pipeline CI/CD (GitHub Actions / GitLab CI)

```

stages:
  - test-unit
  - test-integration
  - build
  - test-e2e
  - deploy-staging
  - test-load
  - deploy-production

test-unit:
  stage: test-unit
  script:
    - npm run test:unit
    - npm run test:coverage

test-integration:
  stage: test-integration
  script:
    - docker-compose up -d postgres redis
    - npm run test:integration

test-e2e:
  stage: test-e2e
  script:
    - npm run test:e2e

test-load:
  stage: test-load
  script:
    - k6 run scripts/load-test.js
  only:
    - main
  when: manual

```

18.5 Resumen de Responsabilidades por Grupo

Grupo	Enfoque	Pruebas Principales	Herramientas Clave
Grupo 1	Portal Web	Unitarias, E2E, Responsive, Accesibilidad	Jest, Playwright, Lighthouse
Grupo 2	Gestión Deportiva	Unitarias, Integración, Funcionales, UAT	Jest, Supertest, Postman
Grupo 3	Servicios	Unitarias, Integración, Seguridad, Carga, Backup	Jest, k6, OWASP ZAP